INTERVIEW PREP — JAVASCRIPT

DSA String Problems

A working cheat sheet of the string questions that come up again and again in coding interviews

20 problems grouped by the technique they teach — not by difficulty — because recognizing *which pattern applies* is the actual interview skill. Each entry has the one-line insight that unlocks it, complexity, and a clean, commented JavaScript solution.

BEFORE YOU CODE — ASK THIS FIRST

- ASCII only, or full Unicode?
- Case-sensitive comparison, or not?
- Can the string be empty? null / undefined?
- Is O(1) / in-place required, or is extra space fine?
- Is the input mutable, or must I return a new string?
- Do duplicate characters need special handling here?

PATTERN RECOGNITION

“palindrome”	**Two pointers from the ends, or expand-around-center**
“longest / shortest substring” + a constraint	**Sliding window**
“anagram” or “same characters”	**Frequency array / hash map**
Brackets, tags, nested “valid” structure	**Stack**
Compare or transform two strings	**2D dynamic programming**
Find / search a string inside another	**Pattern matching (brute force → KMP)**
“in-place” or O(1) extra space required	**Two pointers on the same array**

CONTENTS

01 Two Pointers

Scan from both ends of a string inward, or move two indices at different rates. The classic way to get O(1) extra space.

1. Reverse String EASY

LeetCode 344

Given a string as an array of characters, reverse it in place.

> **Key insight:** The template two-pointer problem: swap the outer characters and move both pointers inward until they meet. This exact shape reappears everywhere.

Time: O(n) **Space:** O(1)

```
function reverseString(s) {
  let left = 0;
  let right = s.length - 1;
  while (left < right) {
    [s[left], s[right]] = [s[right], s[left]];
    left++;
    right--;
  }
  return s;
}

// reverseString(['h','e','l','l','o'])
// -> ['o','l','l','e','h']
```

2. Valid Palindrome EASY

LeetCode 125

Determine if a string is a palindrome, considering only alphanumeric characters and ignoring case.

> **Key insight:** Same two-pointer shape, but skip non-alphanumeric characters before comparing. Read the problem's exact definition of a 'valid' character closely.

Time: O(n) **Space:** O(1)

```
function isPalindrome(s) {
  const isAlnum = (c) => /[a-z0-9]/i.test(c);
  let left = 0;
  let right = s.length - 1;

  while (left < right) {
    if (!isAlnum(s[left])) { left++; continue; }
    if (!isAlnum(s[right])) { right--; continue; }
    if (s[left].toLowerCase() !== s[right].toLowerCase()) {
      return false;
    }
    left++;
    right--;
  }
  return true;
}
```

3. Is Subsequence EASY

LeetCode 392

Given strings s and t, check whether s is a subsequence of t.

> **Key insight:** Two pointers moving at different rates: only advance the s-pointer on a match, but always advance the t-pointer.

Time: O(n) — n = t.length **Space:** O(1)

```
function isSubsequence(s, t) {
  let i = 0;
  let j = 0;
  while (i < s.length && j < t.length) {
    if (s[i] === t[j]) i++;
    j++;
  }
  return i === s.length;
}
```

02 Sliding Window

For 'longest / shortest substring satisfying X' problems. Maintain a window [start, end] and grow or shrink it based on a condition, instead of re-checking every substring.

4. Longest Substring Without Repeating Characters MEDIUM

LeetCode 3

Find the length of the longest substring without repeating characters.

> **Key insight:** Store the last-seen index of each character in a map. On a repeat, jump 'start' directly past the previous occurrence instead of shrinking one step at a time.

Time: O(n) **Space:** O(min(n, charset))

```
function lengthOfLongestSubstring(s) {
  const lastSeen = new Map();
  let start = 0;
  let maxLen = 0;

  for (let end = 0; end < s.length; end++) {
    const c = s[end];
    if (lastSeen.has(c) && lastSeen.get(c) >= start) {
      start = lastSeen.get(c) + 1;
    }
    lastSeen.set(c, end);
    maxLen = Math.max(maxLen, end - start + 1);
  }
  return maxLen;
}
```

5. Longest Repeating Character Replacement MEDIUM

LeetCode 424

Find the longest substring where you can replace at most k characters so every character in it is the same.

> **Key insight:** The window never needs to shrink. Track a running maxFreq (it's fine if it goes stale) — if window size minus maxFreq exceeds k, slide both edges forward instead of shrinking. Window length only ever grows or stays the same.

Time: O(n) **Space:** O(1) — 26 letters

```
function characterReplacement(s, k) {
  const freq = new Array(26).fill(0);
  let start = 0;
  let maxFreq = 0;

  for (let end = 0; end < s.length; end++) {
    const idx = s.charCodeAt(end) - 65; // 'A'
    freq[idx]++;
    maxFreq = Math.max(maxFreq, freq[idx]);

    if (end - start + 1 - maxFreq > k) {
      freq[s.charCodeAt(start) - 65]--;
      start++;
    }
  }
  return s.length - start;
}
```

6. Minimum Window Substring HARD

LeetCode 76

Find the smallest substring of s that contains every character of t, with correct multiplicity.

> **Key insight:** Two frequency maps: 'need' (fixed, from t) and 'window' (grows with end). Expand end until the window is valid, then greedily shrink from start while it stays valid, recording the best length each time.

Time: O(n + m) **Space:** O(m) — m = t.length

```
function minWindow(s, t) {
  if (t.length > s.length) return "";

  const need = new Map();
  for (const c of t) {
    need.set(c, (need.get(c) || 0) + 1);
  }

  const required = need.size;
  let formed = 0;
  const windowCounts = new Map();
  let start = 0;
  let minLen = Infinity;
  let minStart = 0;

  for (let end = 0; end < s.length; end++) {
    const c = s[end];
    windowCounts.set(c, (windowCounts.get(c) || 0) + 1);
    if (need.has(c) && windowCounts.get(c) === need.get(c)) {
      formed++;
    }

    while (formed === required) {
      if (end - start + 1 < minLen) {
        minLen = end - start + 1;
        minStart = start;
      }
      const leftChar = s[start];
      windowCounts.set(leftChar, windowCounts.get(leftChar) - 1);
      if (
        need.has(leftChar) &&
        windowCounts.get(leftChar) < need.get(leftChar)
      ) {
        formed--;
      }
      start++;
    }
  }
  return minLen === Infinity
    ? ""
    : s.substring(minStart, minStart + minLen);
}
```

03 Hashing & Frequency Counting

Use when order doesn't matter but character composition does — anagrams, character counts, uniqueness checks.

7. Valid Anagram EASY

LeetCode 242

Determine whether t is an anagram of s.

> **Key insight:** Same characters, same counts. A fixed 26-length array is faster than a Map when the alphabet is known and small.

Time: O(n) **Space:** O(1)

```
function isAnagram(s, t) {
  if (s.length !== t.length) return false;

  const counts = new Array(26).fill(0);
  for (let i = 0; i < s.length; i++) {
    counts[s.charCodeAt(i) - 97]++; // 'a'
    counts[t.charCodeAt(i) - 97]--;
  }
  return counts.every((c) => c === 0);
}
```

8. Group Anagrams MEDIUM

LeetCode 49

Group an array of strings into sets of anagrams of each other.

> **Key insight:** Anagrams share a canonical form. Sorting a string's characters (or building a 26-count signature) gives a perfect hash-map key.

Time: O(n * k log k) **Space:** O(n * k)

```
function groupAnagrams(strs) {
  const groups = new Map();

  for (const s of strs) {
    const key = s.split("").sort().join("");
    if (!groups.has(key)) groups.set(key, []);
    groups.get(key).push(s);
  }
  return Array.from(groups.values());
}
```

9. First Unique Character in a String EASY

LeetCode 387

Return the index of the first character that never repeats. Return -1 if none exists.

> **Key insight:** Two linear passes beat trying to do it in one: count everything first, then scan again for the first count-of-1.

Time: O(n) **Space:** O(1) — bounded alphabet

```
function firstUniqChar(s) {
  const counts = new Map();
  for (const c of s) {
    counts.set(c, (counts.get(c) || 0) + 1);
  }

  for (let i = 0; i < s.length; i++) {
    if (counts.get(s[i]) === 1) return i;
  }
  return -1;
}
```

04 Stack-Based

Reach for a stack whenever you need to find or match the most recent unmatched thing — brackets, nested structures, cancellation.

10. Valid Parentheses EASY

LeetCode 20

Given a string of brackets, determine if every bracket is closed and correctly nested.

> **Key insight:** Push openers. On a closer, pop and check it matches — a mismatch or an empty stack means invalid immediately.

Time: O(n) **Space:** O(n)

```
function isValid(s) {
  const stack = [];
  const pairs = { ")": "(", "]": "[", "}": "{" };

  for (const c of s) {
    if (c === "(" || c === "[" || c === "{") {
      stack.push(c);
    } else if (stack.pop() !== pairs[c]) {
      return false;
    }
  }
  return stack.length === 0;
}
```

11. Decode String MEDIUM

LeetCode 394

Decode a string like "3[a2[c]]" into "accaccacc", where k[sub] means sub repeats k times.

> **Key insight:** Nested structure means a stack. Every time you enter a '[', push the string built so far and the repeat count; every time you exit a ']', pop and repeat-and-append.

Time: O(n * maxK) **Space:** O(n)

```
function decodeString(s) {
  const countStack = [];
  const strStack = [];
  let current = "";
  let num = 0;

  for (const c of s) {
    if (!isNaN(c)) {
      num = num * 10 + Number(c);
    } else if (c === "[") {
      countStack.push(num);
      strStack.push(current);
      num = 0;
      current = "";
    } else if (c === "]") {
      const repeat = countStack.pop();
      current = strStack.pop() + current.repeat(repeat);
    } else {
      current += c;
    }
  }
  return current;
}
```

12. Remove All Adjacent Duplicates In String EASY

LeetCode 1047

Repeatedly remove pairs of adjacent, identical characters until none remain.

> **Key insight:** A stack simulates the whole cancellation process in a single pass — push a character, unless it matches the top of the stack, in which case pop instead.

Time: O(n) **Space:** O(n)

```
function removeDuplicates(s) {
  const stack = [];
  for (const c of s) {
    if (stack.length && stack[stack.length - 1] === c) {
      stack.pop();
    } else {
      stack.push(c);
    }
  }
  return stack.join("");
}
```

05 Dynamic Programming

Use when comparing or transforming two strings, or when a substring's answer depends cleanly on smaller substrings' answers.

13. Longest Palindromic Substring MEDIUM

LeetCode 5

Find the longest substring that reads the same forward and backward.

> **Key insight:** Every palindrome has a center. Try all 2n-1 possible centers (each character, and each gap between two characters) and expand outward while both sides match.

Time: O(n^2) **Space:** O(1)

```
function longestPalindrome(s) {
  if (s.length < 1) return "";
  let start = 0;
  let maxLen = 1;

  const expand = (left, right) => {
    while (
      left >= 0 &&
      right < s.length &&
      s[left] === s[right]
    ) {
      left--;
      right++;
    }
    return right - left - 1;
  };

  for (let i = 0; i < s.length; i++) {
    const len1 = expand(i, i);     // odd length
    const len2 = expand(i, i + 1); // even length
    const len = Math.max(len1, len2);
    if (len > maxLen) {
      maxLen = len;
      start = i - Math.floor((len - 1) / 2);
    }
  }
  return s.substring(start, start + maxLen);
}
```

14. Longest Common Subsequence MEDIUM

LeetCode 1143

Find the length of the longest subsequence common to two strings (not necessarily contiguous).

> **Key insight:** Classic 2D grid: on a character match, extend the diagonal by one. Otherwise take the best of dropping a character from either string.

Time: O(m * n) **Space:** O(m * n)

```
function longestCommonSubsequence(text1, text2) {
  const m = text1.length;
  const n = text2.length;
  const dp = Array.from({ length: m + 1 }, () =>
    new Array(n + 1).fill(0)
  );

  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      if (text1[i - 1] === text2[j - 1]) {
        dp[i][j] = dp[i - 1][j - 1] + 1;
      } else {
        dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
      }
    }
  }
  return dp[m][n];
}
```

15. Edit Distance HARD

LeetCode 72

Find the minimum number of insert / delete / replace operations to turn word1 into word2.

> **Key insight:** Same grid shape as LCS, but every mismatched cell now considers three operations instead of two, and the first row/column are initialized to i and j (pure insertions or deletions).

Time: O(m * n) **Space:** O(m * n)

```
function minDistance(word1, word2) {
  const m = word1.length;
  const n = word2.length;
  const dp = Array.from({ length: m + 1 }, () =>
    new Array(n + 1).fill(0)
  );

  for (let i = 0; i <= m; i++) dp[i][0] = i;
  for (let j = 0; j <= n; j++) dp[0][j] = j;

  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      if (word1[i - 1] === word2[j - 1]) {
        dp[i][j] = dp[i - 1][j - 1];
      } else {
        dp[i][j] = 1 + Math.min(
          dp[i - 1][j],     // delete
          dp[i][j - 1],     // insert
          dp[i - 1][j - 1]  // replace
        );
      }
    }
  }
  return dp[m][n];
}
```

06 String Manipulation & Parsing

Careful, methodical index-walking. These are less about a clever algorithm and more about correctly handling every edge case, in the right order.

16. String to Integer (atoi) MEDIUM

LeetCode 8

Convert a string to a 32-bit signed integer, following the standard atoi parsing rules.

> **Key insight:** The algorithm is trivial; the edge cases are the problem. Handle, strictly in order: leading whitespace, an optional sign, digits, then clamp to the 32-bit range.

Time: O(n) **Space:** O(1)

```
function myAtoi(s) {
  const n = s.length;
  const INT_MAX = 2 ** 31 - 1;
  const INT_MIN = -(2 ** 31);
  let i = 0;

  while (i < n && s[i] === " ") i++;

  let sign = 1;
  if (i < n && (s[i] === "+" || s[i] === "-")) {
    sign = s[i] === "-" ? -1 : 1;
    i++;
  }

  let result = 0;
  while (i < n && s[i] >= "0" && s[i] <= "9") {
    result = result * 10 + (s.charCodeAt(i) - 48);
    if (sign * result >= INT_MAX) return INT_MAX;
    if (sign * result <= INT_MIN) return INT_MIN;
    i++;
  }
  return sign * result;
}
```

17. Reverse Words in a String MEDIUM

LeetCode 151

Reverse the order of the words in a string, collapsing any extra whitespace.

> **Key insight:** In JS, split/reverse/join solves it in one line. As a follow-up, know the O(1)-extra-space trick from C++/Java: reverse the whole string first, then reverse each word back in place.

Time: O(n) **Space:** O(n)

```
function reverseWords(s) {
  return s.trim().split(/\s+/).reverse().join(" ");
}
```

18. String Compression MEDIUM

LeetCode 443

Compress a character array in place using run-length encoding and return the new length.

> **Key insight:** Two pointers at different speeds: 'read' consumes an entire run of identical characters at once, and 'write' lays down the character plus its count (if > 1).

Time: O(n) **Space:** O(1) extra

```
function compress(chars) {
  let write = 0;
  let read = 0;

  while (read < chars.length) {
    const char = chars[read];
    let count = 0;
    while (read < chars.length && chars[read] === char) {
      read++;
      count++;
    }
    chars[write++] = char;
    if (count > 1) {
      for (const digit of String(count)) {
        chars[write++] = digit;
      }
    }
  }
  return write;
}
```

07 Pattern Matching

Finding one string inside another. Know the brute-force approach cold — it's a perfectly good first answer — and know that KMP exists as the optimized follow-up.

19. Implement strStr() EASY

LeetCode 28

Return the index of the first occurrence of needle in haystack, or -1 if it doesn't occur.

> **Key insight:** The brute-force nested loop is O(n*m) and is a completely fine starting answer. If asked to optimize, that's the cue for KMP below.

Time: O(n * m) **Space:** O(1)

```
function strStr(haystack, needle) {
  const n = haystack.length;
  const m = needle.length;
  if (m === 0) return 0;

  for (let i = 0; i <= n - m; i++) {
    let j = 0;
    while (j < m && haystack[i + j] === needle[j]) {
      j++;
    }
    if (j === m) return i;
  }
  return -1;
}
```

Optimized follow-up — Knuth-Morris-Pratt, O(n + m)

Precompute an LPS ('longest proper prefix that is also a suffix') table for the needle, so on a mismatch you skip ahead instead of restarting from scratch.

```
function strStrKMP(haystack, needle) {
  if (needle.length === 0) return 0;
  const lps = buildLPS(needle);
  let i = 0;
  let j = 0;

  while (i < haystack.length) {
    if (haystack[i] === needle[j]) {
      i++;
      j++;
      if (j === needle.length) return i - j;
    } else if (j > 0) {
      j = lps[j - 1];
    } else {
      i++;
    }
  }
  return -1;
}

function buildLPS(pattern) {
  const lps = new Array(pattern.length).fill(0);
  let len = 0;
  let i = 1;

  while (i < pattern.length) {
    if (pattern[i] === pattern[len]) {
      lps[i++] = ++len;
    } else if (len > 0) {
      len = lps[len - 1];
    } else {
      lps[i++] = 0;
    }
  }
  return lps;
}
```

20. Longest Common Prefix EASY

LeetCode 14

Find the longest string prefix shared by every string in an array.

> **Key insight:** Start with the first string as your candidate prefix, then shrink it from the end every time it fails to match the next string.

Time: O(S) — S = total characters **Space:** O(1)

```javascript
function longestCommonPrefix(strs) {
  if (!strs.length) return "";

  let prefix = strs[0];
  for (let i = 1; i < strs.length; i++) {
    while (!strs[i].startsWith(prefix)) {
      prefix = prefix.slice(0, -1);
      if (!prefix) return "";
    }
  }
  return prefix;
}
```

Quick Complexity Reference

#	PROBLEM	TIME	SPACE
1	Reverse String	O(n)	O(1)
2	Valid Palindrome	O(n)	O(1)
3	Is Subsequence	O(n) — n = t.length	O(1)
4	Longest Substring Without Repeating Characters	O(n)	O(min(n, charset))
5	Longest Repeating Character Replacement	O(n)	O(1) — 26 letters
6	Minimum Window Substring	O(n + m)	O(m) — m = t.length
7	Valid Anagram	O(n)	O(1)
8	Group Anagrams	O(n * k log k)	O(n * k)
9	First Unique Character in a String	O(n)	O(1) — bounded alphabet
10	Valid Parentheses	O(n)	O(n)
11	Decode String	O(n * maxK)	O(n)
12	Remove All Adjacent Duplicates In String	O(n)	O(n)
13	Longest Palindromic Substring	O(n^2)	O(1)
14	Longest Common Subsequence	O(m * n)	O(m * n)
15	Edit Distance	O(m * n)	O(m * n)
16	String to Integer (atoi)	O(n)	O(1)
17	Reverse Words in a String	O(n)	O(n)
18	String Compression	O(n)	O(1) extra
19	Implement strStr()	O(n * m)	O(1)
20	Longest Common Prefix	O(S) — S = total characters	O(1)

How to use this before an interview: cover the code, read only the problem statement and the key insight, and try to reproduce the solution from that alone. If you get stuck, that gap is exactly what to drill next — not the syntax, the pattern recognition.